



创新创业实践

AES-128 软件实现、指令集优化与
CTR/GCM/XTS 工作模式性能实验

实验成员：董胥宏

学 号：202300460176

实验平台：Ubuntu 22.04.5 LTS (VMware)

完成时间：2026 年 7 月 29 日

摘要

本实验围绕对称密码算法的软件实现与性能优化展开，选取 AES-128 作为研究对象，从逐字节基础实现出发，依次完成 T-table、Shuffle/SSSE3、AES-NI 以及 VAES/AVX2 多分组并行实现。实验使用标准已知答案测试验证各版本的加密与解密正确性，并通过反汇编结果确认程序中实际生成了 PSHUFB、AESENC/AESDEC、VAESENC/VAESDEC 以及 VPCLMULQDQ 等指令。

在统一的单线程性能测试条件下，基础 AES-128 的吞吐率为 95.98 MiB/s；T-table、AES-NI 和 VAES/AVX2 分别达到 345.44、5113.01 和 5972.47 MiB/s，对基础版本的加速比分别为 3.60、53.27 和 62.23。Shuffle/SSSE3 版本由于 S 盒仍采用逐字节查表，并引入寄存器与内存之间的数据搬运，吞吐率为 80.07 MiB/s，低于基础版本。该结果说明，局部使用 SIMD 指令并不必然带来整体性能提升。

在分组密码工作模式方面，本实验实现并优化了 AES-CTR、AES-GCM 和 AES-XTS。CTR 使用 AES-NI 与 VAES 八分组并行，最高达到 1950.83 MiB/s；GCM 使用 PCLMULQDQ 加速 GHASH，吞吐率由 10.65 MiB/s 提高到 17.22 MiB/s；XTS 使用 AES-NI 八分组流水线，吞吐率由 80.05 MiB/s 提高到 1383.44 MiB/s。GCM 的认证标签篡改测试成功被拒绝，XTS 的完整分组与密文窃取结果均与 OpenSSL 一致。最终统一检查中，所有正确性测试、指令集证据和性能结果文件均通过验证。

关键词：AES-128；T-table；Shuffle；AES-NI；VAES；CTR；GCM；XTS；PCLMULQDQ

目录

1	实验背景	1
2	实验目的	1
3	实验环境	2
4	AES-128 算法与基础实现	3
4.1	状态表示与轮结构	3
4.2	密钥扩展	3
4.3	基础软件实现验证	3
5	AES 软件优化实现	4
5.1	T-table 优化	4
5.2	Shuffle/SSSE3 优化	5

5.3	AES-NI 优化	6
5.4	VAES/AVX2 双分组并行	7
6	AES 分组加密性能测试	8
6.1	测试方法	8
6.2	结果分析	9
7	AES-CTR 工作模式实现与优化	10
7.1	工作原理	10
7.2	正确性验证	10
7.3	性能结果	11
8	AES-GCM 工作模式实现与优化	11
8.1	GCM 组成	11
8.2	正确性与认证测试	12
8.3	性能结果	13
9	AES-XTS 工作模式实现与优化	13
9.1	工作原理	13
9.2	正确性与 OpenSSL 交叉验证	14
9.3	性能结果	15
10	最终统一检查	15
11	安全性与实现限制分析	17
12	实验结论	17

1 实验背景

对称密码算法广泛用于网络通信、文件保护、磁盘加密和认证加密等场景。AES 的分组长为 128 位，AES-128 使用 128 位密钥并执行 10 轮变换。在软件实现中，不同的状态表示、查表方法、SIMD 向量化方式以及专用密码指令，会显著影响执行效率。

本次作业要求从基本实现出发，对 SM4、AES、GIFT 或 TWINE 中的一种算法进行软件优化，至少覆盖 T-table、Shuffle 以及较新指令集中的两种方法；同时需要基于加解密实现，对 CTR、GCM、XTS 工作模式进行软件优化。为便于使用标准测试向量、硬件密码指令和成熟参考实现进行交叉验证，本实验选择 AES-128。

2 实验目的

1. 独立实现 AES-128 密钥扩展、加密与解密流程；
2. 理解 T-table 将多个轮变换合并为查表与异或的原理；
3. 使用 SSSE3 的 PSHUFB 指令实现 ShiftRows 等字节重排；
4. 使用 AES-NI 和 VAES 实现专用指令加速与多分组并行；
5. 在统一条件下测试五种 AES 实现的吞吐率和加速比；
6. 实现并优化 CTR、GCM、XTS 三种工作模式；
7. 使用标准向量、OpenSSL 和反汇编结果验证正确性与实现真实性；
8. 分析各优化方法的性能收益、安全限制和实际瓶颈。

3 实验环境

表 1 实验环境与处理器指令集

项目	配置
操作系统	Ubuntu 22.04.5 LTS
虚拟化平台	VMware, 4 个虚拟 CPU
处理器	Intel Core Ultra 7 155H
GCC	11.4.0
CMake	3.22.1
OpenSSL	3.5.6
编程语言	C11
支持指令集	SSSE3、AES、PCLMULQDQ、AVX、AVX2、VAES、VPCLMULQDQ
工作目录	/home/xingye/symmetric-crypto-hw3

```
===== HOMEWORK 3 ENVIRONMENT =====
Current directory: /home/xingye/symmetric-crypto-hw3

----- Operating system -----
PRETTY_NAME="Ubuntu 22.04.5 LTS"

----- Compiler and tools -----
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
cmake version 3.22.1
OpenSSL 3.5.6 7 Apr 2026 (Library: OpenSSL 3.5.6 7 Apr 2026)

----- CPU information -----
Architecture:          x86_64
CPU(s):                4
Model name:            Intel(R) Core(TM) Ultra 7 155H
Hypervisor vendor:    VMware

----- Required instruction sets -----
ssse3      : YES
aes        : YES
pclmulqdq  : YES
avx        : YES
avx2       : YES
vaes       : YES
vpclmulqdq : YES
```

图 1 Ubuntu 实验环境、处理器信息与指令集支持情况

图中可见，虚拟机已经向客户机暴露本实验需要的全部指令集，因此可以在同一环境中完成基础实现、传统查表优化、SIMD 优化和专用密码指令优化。

4 AES-128 算法与基础实现

4.1 状态表示与轮结构

AES 将 16 字节输入按照列优先方式排列为一个 4×4 字节状态矩阵：

$$S = \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix}.$$

AES-128 首先执行一次 AddRoundKey，随后执行 9 轮完整变换：

$$\text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{MixColumns} \rightarrow \text{AddRoundKey},$$

最后一轮省略 MixColumns。解密过程使用对应的逆变换。

4.2 密钥扩展

16 字节主密钥扩展为 11 个 16 字节轮密钥，共 176 字节。每生成一组新的轮密钥，需要使用 RotWord、SubWord 和轮常量 Rcon。基础版本将密钥扩展结果保存为字节数组，供加密、解密和后续指令集实现共同使用。

4.3 基础软件实现验证

基础版本使用 S 盒数组、有限域乘法、逐字节行移位和逐列 MixColumns，编译时加入 `-fno-tree-vectorize`，避免编译器自动向量化，从而形成可解释的性能基线。

```

===== AES-128 BASELINE RESULT =====
Source file: /home/xingye/symmetric-crypto-hw3/src/aes_ref.c
Executable: /home/xingye/symmetric-crypto-hw3/bin/aes_ref_test

===== AES-128 REFERENCE IMPLEMENTATION TEST =====
Key:                000102030405060708090A0B0C0D0E0F
Plaintext:          00112233445566778899AABBCCDDEEFF
Expected ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Actual ciphertext:   69C4E0D86A7B0430D8CDB78070B4C55A
Recovered plaintext: 00112233445566778899AABBCCDDEEFF

Encryption KAT: PASS
Decryption KAT: PASS
Overall result: ALL TESTS PASSED

```

图 2 基础 AES-128 软件实现的标准测试向量结果

测试使用密钥

000102030405060708090A0B0C0D0E0F

和明文

00112233445566778899AABBCCDDEEFF.

程序输出密文

69C4E0D86A7B0430D8CDB78070B4C55A,

与标准答案完全一致，解密后也恢复原始明文。

5 AES 软件优化实现

5.1 T-table 优化

T-table 将 SubBytes、ShiftRows 和 MixColumns 的主要计算预先组合为四张 256 项的 32 位查找表 T_0, T_1, T_2, T_3 。普通轮可表示为四次查表结果与轮密钥的异或。例如一个输出字可写为

$$t_0 = T_0[s_0^{(0)}] \oplus T_1[s_1^{(1)}] \oplus T_2[s_2^{(2)}] \oplus T_3[s_3^{(3)}] \oplus k_0.$$

最后一轮仍单独执行 S 盒和行移位，因为该轮没有 MixColumns。

```
===== AES-128 T-TABLE IMPLEMENTATION TEST =====
Key:                000102030405060708090A0B0C0D0E0F
Plaintext:          00112233445566778899AABBCCDDEEFF
Expected ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
T-table ciphertext:  69C4E0D86A7B0430D8CDB78070B4C55A
Recovered plaintext: 00112233445566778899AABBCCDDEEFF

T-table generation: SUCCESS
Encryption KAT: PASS
Decryption KAT: PASS
Overall result: ALL TESTS PASSED
```

图 3 AES-128 T-table 实现的加密与解密正确性结果

T-table 版本的实际密文与标准答案一致，加密和解密测试均通过。该方法能减少有限域乘法和逐步骤状态变换，但查表地址依赖秘密中间值，存在缓存侧信道风险，因此本实验仅将其作为传统软件优化方案进行比较。

5.2 Shuffle/SSSE3 优化

Shuffle 版本使用 SSSE3 的 `_mm_shuffle_epi8()`，其对应指令为 `PSHUFB`。通过固定掩码，一条指令即可完成 16 字节状态的 `ShiftRows`；同时使用 128 位 SIMD 寄存器并行计算四列 `MixColumns`。

```
1 const __m128i mask = _mm_setr_epi8(
2     0,  5, 10, 15,
3     4,  9, 14,  3,
4     8, 13,  2,  7,
5     12, 1,  6, 11
6 );
7 state = _mm_shuffle_epi8(state, mask);
```



```

===== AES-128 SHUFFLE/SSSE3 RESULT =====
===== AES-128 SHUFFLE/SSSE3 IMPLEMENTATION TEST =====
SSSE3 runtime support: YES

Key:                000102030405060708090A0B0C0D0E0F
Plaintext:          00112233445566778899AABBCCDDEEFF
Expected ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Reference ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Shuffle ciphertext:  69C4E0D86A7B0430D8CDB78070B4C55A
Recovered plaintext: 00112233445566778899AABBCCDDEEFF

Encryption KAT: PASS
Decryption KAT: PASS
Matches reference: PASS
Overall result: ALL TESTS PASSED

----- PSHUFB instruction evidence -----
1496:      66 41 0f 38 00 f8      pshufb xmm7,xmm8
14ab:      66 44 0f 38 00 d5      pshufb xmm10,xmm5
14b1:      66 44 0f 38 00 da      pshufb xmm11,xmm2
14cf:      66 0f 38 00 f3      pshufb xmm6,xmm3

```

图 4 Shuffle/SSSE3 实现及 PSHUFB 指令证据

反汇编结果中出现多条 `pshufb` 指令，证明程序确实使用了 `Shuffle` 指令。该版本的 S 盒仍通过逐字节数组查找实现，因此需要把寄存器内容暂存到内存、查表后重新装载，这一开销将在性能结果中体现。

5.3 AES-NI 优化

AES-NI 为 AES 轮函数提供专用指令。AESENC 完成普通加密轮，AESENCLAST 完成最后一轮；解密使用 AESDEC 和 AESDECLAST，AESIMC 用于变换中间解密轮密钥。

```

===== AES-128 AES-NI RESULT =====
===== AES-128 AES-NI IMPLEMENTATION TEST =====
AES-NI runtime support: YES

Key:                000102030405060708090A0B0C0D0E0F
Plaintext:          00112233445566778899AABBCCDDEEFF
Expected ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Reference ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
AES-NI ciphertext:   69C4E0D86A7B0430D8CDB78070B4C55A
Recovered plaintext: 00112233445566778899AABBCCDDEEFF

Encryption KAT: PASS
Decryption KAT: PASS
Matches reference: PASS
Overall result: ALL TESTS PASSED

----- AES-NI instruction evidence -----
12dc:      66 41 0f 38 dc c3      aesenc xmm0,xmm11
130e:      66 0f 38 dd c3      aesenclast xmm0,xmm3
131f:      66 41 0f 38 de c6      aesdec xmm0,xmm14
135f:      66 0f 38 df c2      aesdeclast xmm0,xmm2
1232:      66 45 0f 38 db da      aesimc xmm11,xmm10

```

图 5 AES-NI 实现正确性及 AESENC、AESDEC 等指令证据

截图中加密、解密和基础版本交叉比较均通过,反汇编同时出现 `aesenc`、`aesenclast`、`aesdec`、`aesdeclast` 和 `aesimc`。

5.4 VAES/AVX2 双分组并行

VAES 将 AES 轮操作扩展到 256 位或 512 位向量寄存器。本实验使用 256 位 ymm 寄存器,每个寄存器包含两个相互独立的 128 位 AES 通道。轮密钥通过广播复制到两个通道,一条 VAESENK 指令即可同时处理两个分组。

```

===== AES-128 VAES/AVX2 RESULT =====
===== AES-128 VAES/AVX2 IMPLEMENTATION TEST =====
VAES runtime support: YES
AVX2 runtime support: YES

Key:                000102030405060708090A0B0C0D0E0F

[Block 0: standard test vector]
Plaintext:          00112233445566778899AABBCCDDEEFF
Expected ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
VAES ciphertext:    69C4E0D86A7B0430D8CDB78070B4C55A

[Block 1: parallel test block]
Plaintext:          6BC1BEE22E409F96E93D7E117393172A
Reference ciphertext: 47C58D5E21CAAF840D015B7D9B910981
VAES ciphertext:    47C58D5E21CAAF840D015B7D9B910981

[Recovered plaintext]
Recovered block 0:   00112233445566778899AABBCCDDEEFF
Recovered block 1:   6BC1BEE22E409F96E93D7E117393172A

Standard vector KAT: PASS
Two-block reference match: PASS
Two-block decryption: PASS
Parallel blocks processed: 2
Overall result: ALL TESTS PASSED

----- VAES instruction evidence -----
13ab:      c4 c2 6d dc d7      vaesenc ymm2,ymm2,ymm15
1403:      c4 c2 6d dd d5      vaesenclast ymm2,ymm2,ymm13
1415:      c4 e2 7d de 84 24 a0 vaesdec ymm0,ymm0,YMMWORD PTR [rsp+0xa0]
1452:      c4 c2 7d df c4      vaesdeclast ymm0,ymm0,ymm12

```

图 6 VAES/AVX2 双分组并行及 VAES 指令证据

两个分组的加密结果均与基础版本一致，解密恢复成功。反汇编中的 `vaesenc`、`vaesenclast`、`vaesdec` 和 `vaesdeclast` 均使用 `ymm` 寄存器。

6 AES 分组加密性能测试

6.1 测试方法

五种实现均在同一虚拟机中以单线程运行，固定在 CPU 0，缓冲区大小为 16 MiB。每种实现执行 3 个计时轮次，每轮至少运行 1 秒，对结果排序后取中位数。测试程序在计时前预热一次，并计算校验和，防止编译器删除加密过程。

```

===== AES-128 BLOCK PERFORMANCE SUMMARY =====
Mode           : single thread
CPU affinity    : CPU 0
Buffer         : 16 MiB
Trials         : 3, median reported

Implementation      Median MiB/s      Speedup
-----
Reference           95.98          1.00x
T-table             345.44          3.60x
Shuffle-SSSE3       80.07          0.83x
AES-NI              5113.01         53.27x
VAES-AVX2           5972.47         62.23x

```

图 7 五种 AES-128 实现的吞吐率与加速比

表 2 AES-128 分组加密性能结果

实现	吞吐率 (MiB/s)	相对基础版加速比
Reference	95.98	1.00
T-table	345.44	3.60
Shuffle-SSSE3	80.07	0.83
AES-NI	5113.01	53.27
VAES-AVX2	5972.47	62.23

6.2 结果分析

T-table 将基础版本提升到 3.60 倍，说明预计算与查表能够显著降低普通 C 实现的轮函数开销。Shuffle 版本只有基础版的 0.83 倍，主要原因是当前实现只向量化了 ShiftRows 与 MixColumns，S 盒仍为逐字节查表，寄存器与内存之间的往返抵消了局部 SIMD 收益。

AES-NI 达到 5113.01 MiB/s，为基础版本的 53.27 倍；VAES/AVX2 进一步达到 5972.47 MiB/s，为基础版本的 62.23 倍，相比 AES-NI 提高约 16.8%。VAES 没有达到两倍加速，是因为循环、加载、存储和轮密钥处理仍占用执行资源，且测试只使用 256 位向量处理两个分组。

7 AES-CTR 工作模式实现与优化

7.1 工作原理

CTR 模式对连续计数器进行 AES 加密，并将输出作为密钥流与数据异或：

$$C_i = P_i \oplus E_K(\text{CTR}_i).$$

解密使用相同运算：

$$P_i = C_i \oplus E_K(\text{CTR}_i).$$

由于各计数器分组互不依赖，CTR 适合使用多分组流水线和 VAES 并行。

7.2 正确性验证

本实验实现基础 CTR、AES-NI 八分组 CTR 和 VAES 八分组 CTR，并使用标准 64 字节测试向量验证。计数器采用 128 位大端递增方式。

```
===== AES-128 CTR CORRECTNESS RESULT =====
===== AES-128 CTR MODE TEST =====
Key:                2B7E151628AED2A6ABF7158809CF4F3C
Initial counter:    F0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF
Plaintext block 0:  6BC1BEE22E409F96E93D7E117393172A
Expected block 0:   874D6191B620E3261BEF6864990DB6CE
Reference block 0:  874D6191B620E3261BEF6864990DB6CE
AES-NI block 0:     874D6191B620E3261BEF6864990DB6CE
VAES block 0:       874D6191B620E3261BEF6864990DB6CE

Reference CTR KAT : PASS
AES-NI CTR KAT    : PASS
VAES CTR KAT      : PASS
CTR decrypt test  : PASS
All 64 bytes match: YES
Overall result     : ALL TESTS PASSED
```

图 8 CTR 基础版、AES-NI 与 VAES 版本的标准向量验证结果

三个实现对全部 64 字节产生完全相同的密文，且再次执行 CTR 运算能够恢复明文。

7.3 性能结果

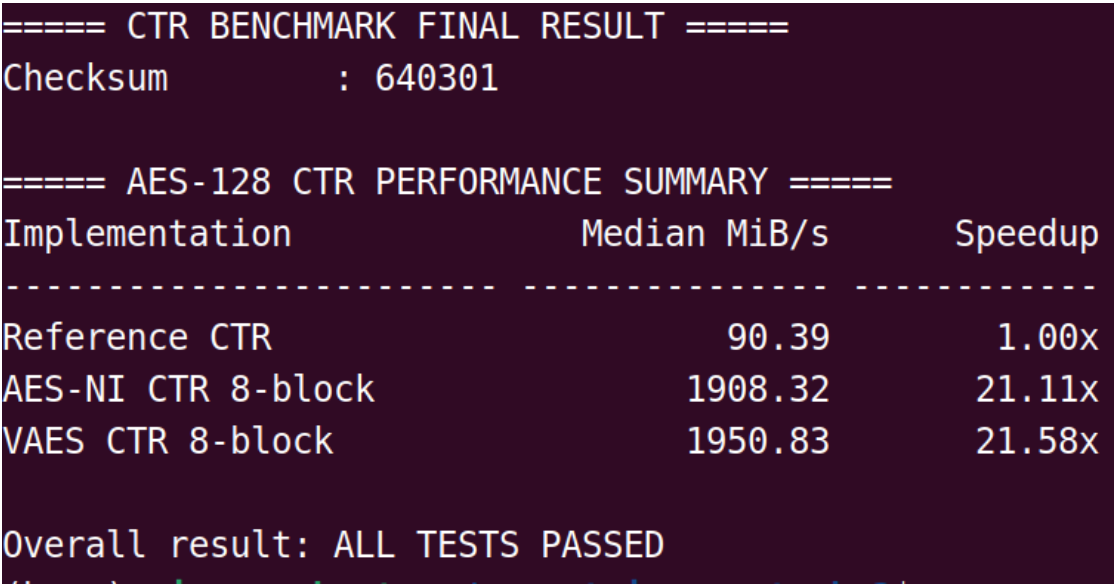


图 9 AES-CTR 三种实现的吞吐率对比

表 3 AES-CTR 性能结果

实现	吞吐率（MiB/s）	相对基础版加速比
Reference CTR	90.39	1.00
AES-NI CTR 8-block	1908.32	21.11
VAES CTR 8-block	1950.83	21.58

CTR 中 AES-NI 和 VAES 均获得约 21 倍加速，但整体加速比低于纯分组加密，因为工作模式还需要生成计数器、与明文异或以及读写输出。VAES 仅比 AES-NI 提高约 2.2%，说明当前逐个构造计数器的普通 C 代码已成为新的瓶颈。

在真实系统中，同一密钥下不得重复使用相同的计数器序列，否则两个密文的异或将暴露两个明文的异或关系。本实验中重复计数器只用于可复现的性能测试。

8 AES-GCM 工作模式实现与优化

8.1 GCM 组成

GCM 是认证加密模式，由 CTR 加密和 GHASH 认证两部分组成：

$$\text{GCM} = \text{CTR} + \text{GHASH}.$$

GHASH 在 $GF(2^{128})$ 上进行多项式乘法，使用不可约多项式

$$x^{128} + x^7 + x^2 + x + 1.$$

本实验分别实现逐位有限域乘法和基于 PCLMULQDQ 的无进位乘法，并使用 AES-NI 完成 CTR 部分。

8.2 正确性与认证测试

测试使用全零密钥、96 位全零 IV 和一个全零明文分组。标准密文为

0388DACE60B6A392F328C2B971B2FE78,

标准标签为

AB6E47D42CEC13BDF53A67B21257BDDF.

```
===== AES-128 GCM CORRECTNESS RESULT =====
AES-NI runtime support : YES
PCLMUL runtime support : YES
SSSE3 runtime support : YES
GF multiply cross-check : PASS
Reference ciphertext KAT: PASS
Reference tag KAT      : PASS
PCLMUL ciphertext KAT  : PASS
PCLMUL tag KAT         : PASS
Authenticated decryption: PASS
Tampered tag rejected  : PASS
Overall result          : ALL TESTS PASSED

----- Actual PCLMUL instruction evidence -----
29a0:      c4 e3 79 44 e9 00      vpcmullqlqdq xmm5,xmm0,xmm1
29a6:      c4 e3 79 44 e1 11      vpclmulhqhdq xmm4,xmm0,xmm1
29ac:      c4 e3 79 44 f1 01      vpclmulhqldq xmm6,xmm0,xmm1
29b2:      c4 e3 79 44 c1 10      vpcmullhqhdq xmm0,xmm0,xmm1
```

图 10 GCM 正确性、标签篡改检测及 VPCLMULQDQ 指令证据

逐位 GHASH 和 PCLMUL GHASH 的密文与标签均匹配标准结果。使用正确标签时认证解密成功；修改标签最后一位后，程序拒绝解密。反汇编结果出现 vpcmullqlqdq 和 vpclmulhqhdq 等无进位乘法指令。

8.3 性能结果

```

===== GCM BENCHMARK FINAL RESULT =====
Checksum           : 135870

===== AES-128 GCM PERFORMANCE SUMMARY =====
Implementation      Median MiB/s      Speedup
-----
Reference GHASH      10.65        1.00x
PCLMUL GHASH         17.22        1.62x

Overall result: ALL TESTS PASSED

```

图 11 GCM 基础 GHASH 与 PCLMUL GHASH 的性能对比

表 4 AES-GCM 性能结果

实现	吞吐率 (MiB/s)	加速比
Reference GHASH	10.65	1.00
PCLMUL GHASH	17.22	1.62

PCLMUL 版本比逐位乘法提高 62%。该加速幅度低于成熟密码库，原因是本实验只用 PCLMUL 完成 128 位无进位乘法，而 256 位中间结果的约简仍采用逐位循环；此外还需要执行字节内位逆序和数据格式转换。因此，有限域约简成为新的主要瓶颈。

GCM 真实使用时必须保证同一密钥下 IV 唯一，并且只能在认证标签验证成功后向上层交付明文。

9 AES-XTS 工作模式实现与优化

9.1 工作原理

AES-128-XTS 使用两把独立的 AES-128 密钥，总密钥长度为 256 位。其中 K_1 用于数据加解密， K_2 用于生成初始 tweak：

$$T_0 = E_{K_2}(I).$$

第 i 个完整分组的加密为

$$C_i = E_{K_1}(P_i \oplus T_i) \oplus T_i,$$

每处理一个分组，tweak 在 $GF(2^{128})$ 中乘以 α 。对于最后一个非完整分组，XTS 使用 Ciphertext Stealing (CTS, 密文窃取)。

9.2 正确性与 OpenSSL 交叉验证

本实验实现基础 XTS 和 AES-NI 八分组流水线 XTS，测试 64 字节完整分组数据和 47 字节非整分组数据。OpenSSL EVP 的 AES-128-XTS 作为独立参考实现。

```
===== AES-128 XTS CORRECTNESS RESULT =====
===== AES-128 XTS MODE TEST =====
AES-NI runtime support : YES
OpenSSL XTS oracle      : ENABLED
Full-block length       : 64 bytes
CTS test length         : 47 bytes

Data key K1:            000102030405060708090A0B0C0D0E0F
Tweak key K2:           101112131415161718191A1B1C1D1E1F
Tweak input:            00000000000000000000000000000001
Full plaintext block 0: 0714212E3B4855626F7C8996A3B0BDCA
OpenSSL ciphertext 0: 4C0F2EC9805E23AB5A9AF5EED6E33FF4
Reference ciphertext 0: 4C0F2EC9805E23AB5A9AF5EED6E33FF4
AES-NI ciphertext 0: 4C0F2EC9805E23AB5A9AF5EED6E33FF4

[Correctness results]
Full reference vs OpenSSL : PASS
Full AES-NI vs OpenSSL    : PASS
CTS reference vs OpenSSL  : PASS
CTS AES-NI vs OpenSSL     : PASS
Reference full decrypt    : PASS
AES-NI full decrypt       : PASS
Reference CTS decrypt     : PASS
AES-NI CTS decrypt        : PASS
XTS authentication       : NOT PROVIDED
Overall result            : ALL TESTS PASSED
```

图 12 XTS 完整分组、CTS 与 OpenSSL 交叉验证结果

基础版与 AES-NI 版在完整分组和 CTS 两种情况下均与 OpenSSL 输出一致，且所有解密测试通过。截图中的 XTS authentication: NOT PROVIDED 是模式本身的性质：XTS 只提供存储数据机密性，不提供完整性认证。

9.3 性能结果

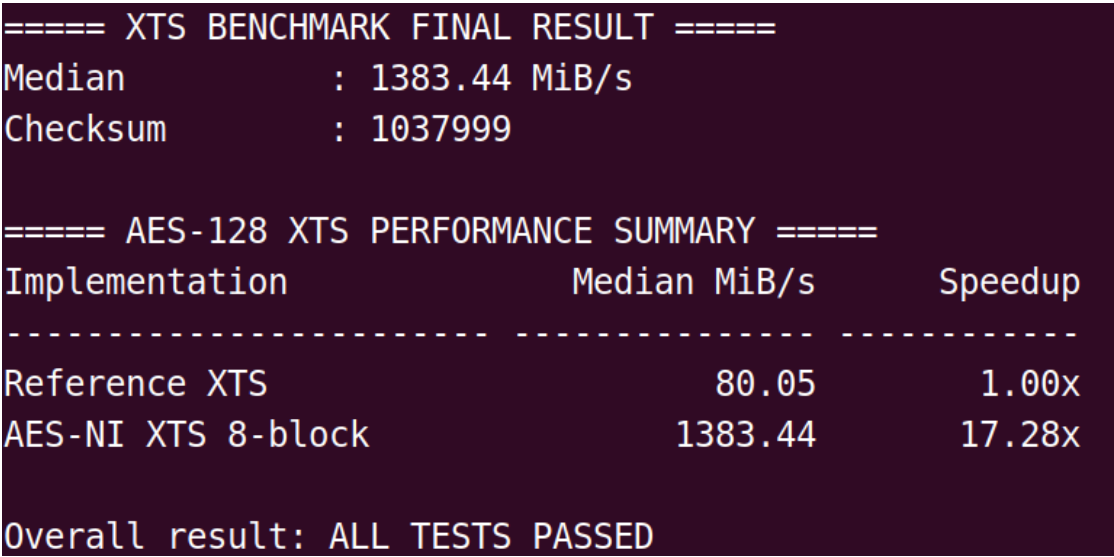


图 13 XTS 基础软件版与 AES-NI 八分组版的性能对比

表 5 AES-XTS 性能结果

实现	吞吐率（MiB/s）	相对基础版加速比
Reference XTS	80.05	1.00
AES-NI XTS 8-block	1383.44	17.28

AES-NI 八分组流水线将 XTS 性能提高到 17.28 倍。其加速比低于纯 AES 分组加密，因为每个数据分组还需要 tweak 乘 α 、加密前后异或 tweak，并处理更多的内存操作。真实磁盘加密中，每个数据单元应使用由其编号派生的不同 tweak 输入。

10 最终统一检查

为避免仅凭单张截图判断实验是否完成，本实验编写统一检查脚本，依次执行八项正确性测试，检查 Shuffle、AES-NI、VAES 和 PCLMUL 指令证据，并确认四组性能结果文件存在。

```
===== HOMEWORK 3 FINAL CHECK =====  
  
----- Correctness tests -----  
[PASS] Reference AES  
[PASS] T-table AES  
[PASS] Shuffle AES  
[PASS] AES-NI AES  
[PASS] VAES AES  
[PASS] CTR mode  
[PASS] GCM mode  
[PASS] XTS mode  
  
----- Instruction evidence -----  
[PASS] Shuffle/SSSE3 instruction evidence  
[PASS] AES-NI encryption instruction evidence  
[PASS] AES-NI decryption instruction evidence  
[PASS] VAES encryption instruction evidence  
[PASS] VAES decryption instruction evidence  
[PASS] PCLMUL/GHASH instruction evidence  
  
----- Performance result files -----  
[PASS] AES block benchmark result file  
[PASS] CTR benchmark result file  
[PASS] GCM benchmark result file  
[PASS] XTS benchmark result file  
  
----- Source files -----  
Source files found: 17  
  
FINAL STATUS: ALL REQUIRED TESTS PASSED
```

图 14 全部算法、工作模式、指令证据与性能文件的统一检查结果

最终结果包括：

- Reference、T-table、Shuffle、AES-NI、VAES 五种 AES 实现全部通过；
- CTR、GCM、XTS 三种工作模式全部通过；
- PSHUFB、AESENC/AESDEC、VAESENC/VAESDEC、PCLMUL 指令证据全部通过；
- AES 分组、CTR、GCM、XTS 四组性能结果文件全部存在；
- 共检查到 17 个 C/H 源文件；

- 最终状态为 ALL REQUIRED TESTS PASSED。

11 安全性与实现限制分析

1. **T-table**: 查表地址依赖秘密状态,可能泄露缓存访问模式,不宜直接作为高安全等级生产实现。
2. **Shuffle**: 本实验用于展示 PSHUFB 和 SIMD 数据重排,未实现完整的 bitslice 或寄存器内 S 盒,因此不能代表 Shuffle AES 的最优性能。
3. **AES-NI/VAES**: 专用轮指令避免传统大表访问,并显著提高吞吐率,但实际安全性仍依赖密钥管理、工作模式参数和完整实现。
4. **CTR**: 同一密钥下计数器序列必须唯一,计数器复用会破坏机密性。
5. **GCM**: IV 必须唯一;标签必须采用常数时间比较;认证失败时不得使用解密输出。
6. **XTS**: 只提供机密性,不提供认证;需要防篡改时必须结合其他完整性保护机制。

12 实验结论

本实验完成了 AES-128 从基础字节实现到传统查表、SIMD 和专用指令集的多层次优化,并对 CTR、GCM、XTS 三种工作模式进行了正确性验证和性能比较。基础实现、T-table、Shuffle、AES-NI 和 VAES 版本均通过标准向量或参考实现验证,反汇编结果确认程序实际生成了对应硬件指令。

性能结果表明, T-table 能够取得 3.60 倍加速,但存在缓存侧信道问题; AES-NI 和 VAES 分别取得 53.27 倍与 62.23 倍加速,是本实验中最有效的 AES 轮函数优化。Shuffle 版本由于 S 盒未向量化,仅达到基础版本的 0.83 倍,说明优化必须从完整数据路径出发,而不能只关注某一条指令。

工作模式实验中, CTR 的最高吞吐率为 1950.83 MiB/s, XTS 的 AES-NI 八分组实现达到 1383.44 MiB/s; PCLMUL GHASH 将 GCM 从 10.65 MiB/s 提高到 17.22 MiB/s。GCM 成功检测标签篡改, XTS 的 CTS 结果与 OpenSSL 一致。最终统一检查显示全部要求均已完成。

综上,本实验不仅验证了不同软件优化技术对 AES 执行效率的影响,也说明工作模式中的计数器、认证、多项式约简、tweak 更新和内存访问可能成为新的瓶颈。专用

指令能够显著提升核心密码运算，但完整系统性能与安全性仍取决于算法、工作模式和实现细节的整体设计。